ARCHITECTURE & OPERATIONS REPORT

Handdy CRM

Updated Architecture and Render Data Optimization Report

Complete source preservation with an evidence-led implementation plan.

Prepared 26 October 2026 | Source architecture dated 26 October 2026

REPORTED METRIC	VALUE	INTERPRETATION
Service-initiated	4.8 GB	Outbound category; destination attribution required
HTTP responses	1.92 GB	Responses to browser and desktop requests
Arithmetic sum	6.72 GB	Approximately 7 GB; verify matching scope and period

Report scope
Includes current and proposed diagrams, all source Markdown, the complete target folder tree, the full Codex prompt, and the original 10-page architecture PDF. No repository, live Render account or database was inspected.

Evidence labels throughout: SOURCE-DOCUMENTED / USER-REPORTED / CALCULATED / HYPOTHESIS / PROPOSED.

READING GUIDE

Contents and source integrity

PART	CONTENTS
01-03	Evidence, current architecture and request / API inventory
04-06	Data model, background jobs and proposed architecture
07-09	Render categories, polling calculations and priorities
10-12	Timer reliability, sync design and security
13-14	Migration, testing and acceptance gates
15	References, limitations and preservation checks
Appendix A	Complete Pasted markdown(4).md, reflowed without omissions; includes the full target tree and Codex prompt
Appendix B	All 10 original PDF pages, retained in their original order

What “confirmed” means in this report

A statement confirmed in a supplied document is confirmed as documentation, not as deployed behavior. Source-documentation claims need repository and production verification before implementation. User-reported usage is not an independently audited billing export.

Source quality

The original PDF is image-based and has no extractable text. Its first page contains a large blank area and a partially visible diagram; several tables and diagrams are clipped horizontally. Appendix B preserves those source artifacts exactly. Clear diagrams and readable inventories in Part I reconstruct only visible, documented information. Cropped content has not been invented.

Appendix A preserves every paragraph, list, table, tree entry, restriction and prompt instruction from the Markdown. Typographic rendering may differ; source wording and order are retained. The intact source PDF is embedded as an attachment as well as appended as pages.

Main decision
Keep the single Render backend, Electron, React, Node.js/Express and MongoDB Atlas. Measure traffic by category and destination, then apply incremental changes with timekeeping and permission regression checks. Additional infrastructure is not justified by the supplied evidence.

01 / EVIDENCE

Facts, calculations and hypotheses

LABEL	WHAT IS ESTABLISHED	WHAT REMAINS OPEN
SOURCE-DOCUMENTED	One Render service; Node + Express 5; Electron 44; React 19 + Vite 8; MongoDB Atlas + Mongoose 9.	Actual installed versions, configuration and deployed behavior.
SOURCE-DOCUMENTED	20 s heartbeat; 30 s desktop refresh; 90 s silence recovery; 13 collections.	Request fan-out, actual payload sizes, retries and runtime query counts.
USER-REPORTED	15 employees; 4.8 GB service-initiated; 1.92 GB HTTP responses since early October.	Exact UTC start/end, service/workspace scope, GB/GiB display and category export.
CALCULATED	4.8 + 1.92 = 6.72 GB; 36,000 heartbeat + refresh requests per 8-hour day under stated assumptions.	These are arithmetic estimates, not measured usage attribution.
HYPOTHESIS	Polling, database traffic, reconnects or external calls may cause avoidable transfer.	No destination or endpoint has been shown to cause the reported 4.8 GB.
PROPOSED	Delta sync, cache validation, query optimization and modularization.	Savings and impact must be demonstrated in the repository and measured after rollout.

Existing optimizations must be audited first

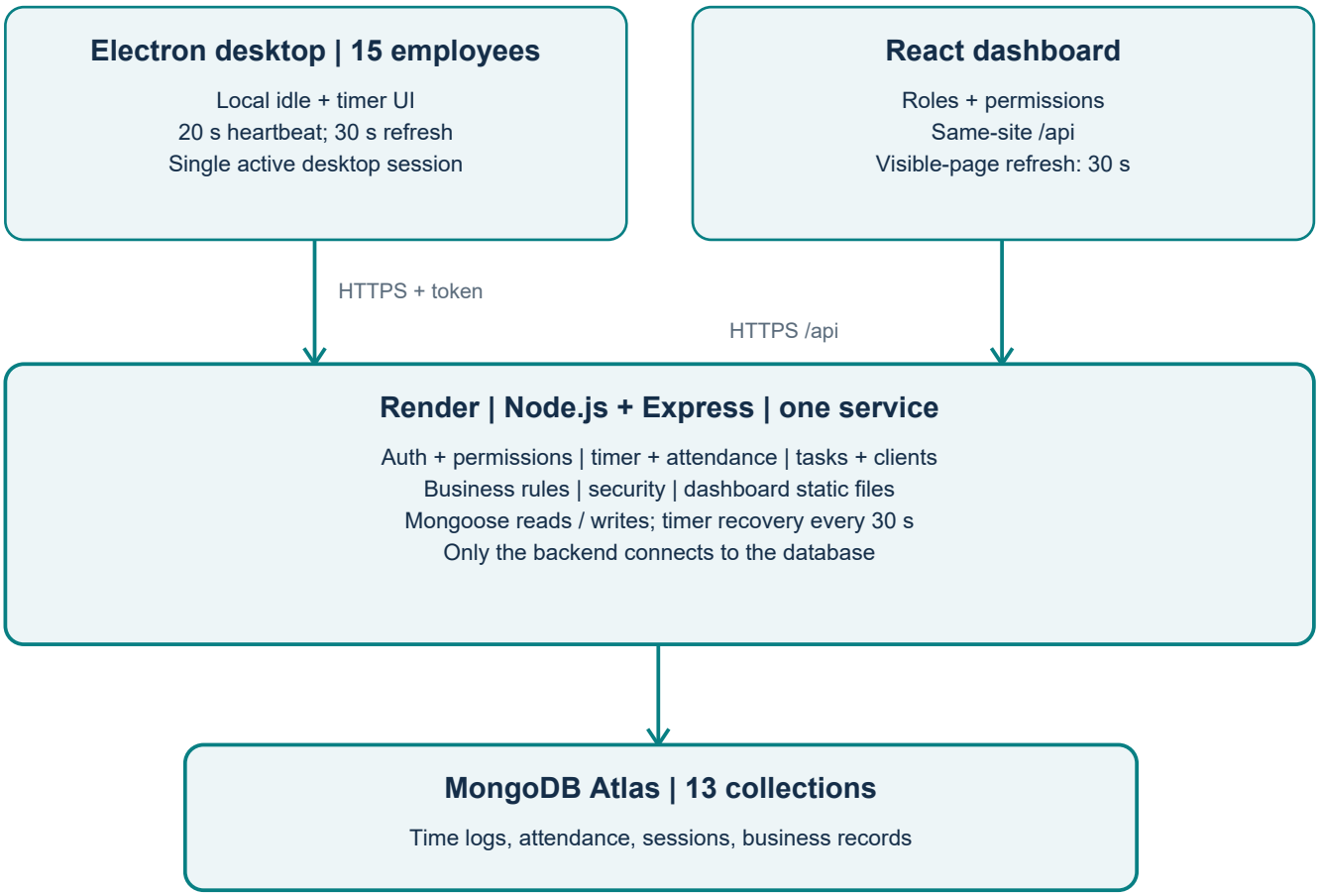
The PDF already describes compression, unchanged catalog responses using 304, a 15-second sign-in cache, a 60-second catalog cache, changed-row dashboard reporting, visible-tab refresh, unique timer/session constraints and server-authoritative time. These are existing documented features; the plan must validate and extend them rather than claim they are all missing.

Evidence needed for attribution

Collect a time-aligned Render category export, endpoint request and response measurements, database command counts, process outbound destination totals, deployment/restart events, updater hosting configuration and workload hours. Record the measurement layer: serialized JSON, compressed body or socket bytes. These quantities are not interchangeable.

02 / CURRENT SYSTEM

Current architecture and request path



SOURCE-DOCUMENTED. Both clients call the same token-protected API. Only the backend accesses MongoDB. The server also serves dashboard files. Desktop time entry is separate from browser reporting and administration. [S1, pp. 1-3]

Delivery paths

A push to GitHub main triggers Render to install dependencies, build the dashboard and restart the server. Desktop changes require a version increase, installer build and GitHub Release with .exe, .blockmap and latest.yml. Desktop checks after 15 seconds and every 6 hours. From documented version 1.2.32, installation restarts the app when no timer runs and the PC has been idle for one minute; otherwise installation occurs on close, then reopens signed in. [S1, pp. 7-8]

Request pipeline

HTTPS /api with Bearer token → compression and 1 MB JSON limit (413) → NoSQL key guard → security headers and CORS → route lookup (404) → protect / account / session validation (401 SESSION_REVOKED or 403 ACCOUNT_INACTIVE) → role or permission gate (403) → controller and services (400 / 409 on business conflict) → Mongoose model and unique constraints → JSON / conditional 304 response. [S1, pp. 2-3]

03 / API INVENTORY

Documented endpoints and callers

All paths below use /api except /health. Permission descriptions are limited to readable source content; do not reconstruct the clipped right column as an authoritative access matrix. [S1, pp. 4-5]

AREA / CALLER	DOCUMENTED PATHS
Sign-in / both	/auth/login; /auth/quick-login; /auth/me; /auth/logout; /auth/login-requests/...
Passwords / dashboard	/auth/forgot-password; /auth/reset-password; /auth/change-password
Employees / dashboard	/employees; /employees/options; /employees/my-team; /teams; /auth/register
Catalogs / both	/clients; /projects; /tasks; /tasks/bulk
Timer / desktop	/time-logs/start; /time-logs/heartbeat; /time-logs/stop; /time-logs/active; /time-logs/note; /time-logs/history
Reports / dashboard	/time-logs/admin; /time-logs/team; /time-logs/live; /time-logs/history
Manual time / dashboard	/time-logs/manual; /time-logs/manual/options; /time-logs/manual/:id
Attendance / dashboard	/attendance/admin/history; /attendance/team/history; /attendance/history
Billing / dashboard	/billing/employees; /billing/report
Settings / both	/settings/idle-policy; /settings/employee-idle-policies
Permissions / dashboard	/permissions/me; /permissions; /permissions/roles/:role
Shifts / dashboard	/shifts; /schedules (source says pages exist, not in menu)
Health / Render	/health (public health check; no /api prefix)

Second-device sequence diagrams abbreviate login-request paths without /auth. Resolve the actual router prefix in code before modifying clients. Password reset email uses SMTP on Render according to the source; transport configuration and failure handling remain unverified.

04 / DATA & ACCESS

Data relationships and business rules

Documented model relationships

TaskTimeLog is central: it references employee, project, task and client and stores start, stop, pauses, worked seconds and idle seconds. Employee holds role, team, permissions and project rates used for billing. Attendance stores a daily record with check-in and check-out derived from timers. Team links to its lead; Schedule links employee, team and Shift. AuthSession links employee and device, distinguishing desktop from web sessions. [S1, p. 8]

VISIBLE MODEL	DOCUMENTED ROLE
Client / Project / Task	Names, status and project / client / team relationships; task is work or break.
TaskTimeLog / Attendance	Worked-time source and per-employee daily attendance.
Employee / Team / Shift / Schedule	Identity, role, billing rates, organizational and shift assignments.
AuthSession	Active / revoked desktop or web session.
Role permissions / system settings	Standalone role switches and idle / logout settings; labels are clipped in the source diagram.
Device login request	Second-computer pending approval flow; model label is clipped.

The source states 13 collections. Ten model labels are fully readable; role permissions, system settings and the device-request label are clipped but identifiable by context. Exact schema / collection names must be checked in models/. Do not create or rename collections from this diagram alone.

Access rules to preserve

The source gives alpha, admin and manager full permission access. Only admin and alpha edit the Permissions page; only alpha sets per-person idle rules. HR, team_lead and employee depend on role switches and personal grants. Role gates, role permissions (documented propagation within 30 seconds) and personal employee-management grants are checked on the server. The dashboard reads /permissions/me to hide controls; that UI behavior does not replace server authorization. [S1, pp. 8-9]

05 / BACKGROUND WORK

Polling and the employee lifecycle

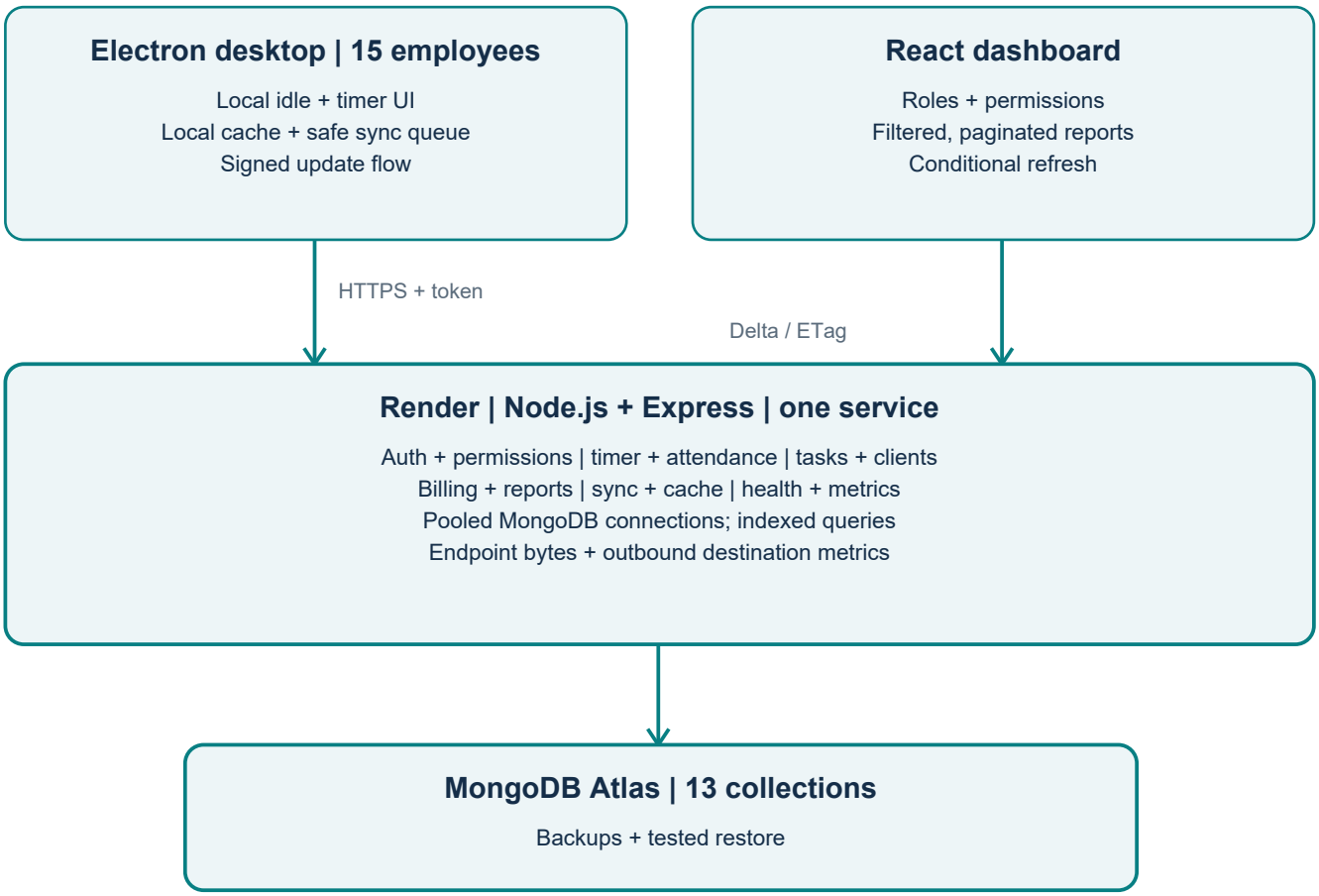
WHERE / INTERVAL	DOCUMENTED BEHAVIOR
Desktop / 5 s	Read keyboard / mouse idle time; stop after 5 minutes idle; sign out after 50 minutes; report lock state. Local detection is not itself a network request.
Desktop / 20 s	Heartbeat keeps the active timer alive.
Desktop / 30 s	Check session; refresh today's timesheet and project / client / task lists. A cycle can fan out into multiple calls.
Desktop / 60 s	Reload idle and logout policy.
Desktop / 15 s then 6 h	Check GitHub Releases for desktop updates.
Clients / while waiting	Long-poll second-device sign-in approval; request expires after 2 minutes.
Dashboard / 30 s	Refresh visible open page; source already says only changed report rows.
Server / 30 s	Close timers silent for 90 s at their last heartbeat and update attendance.
Server / 15 s and 60 s	Sign-in and catalog cache intervals; verify whether these describe TTLs, sweeps or refresh jobs in code.

Employee sequence to retain

Login creates an AuthSession and returns tokens. Starting a project / task / client timer creates a running TaskTimeLog and attendance check-in, unlocking the screen. Heartbeats and history sync run while working. After five minutes idle, the client submits a stop including idleSeconds; the backend applies the existing idle and attendance rules. Crash or lost network invokes server recovery. Closing CRM submits stop before quitting. While no timer runs, the source says desktop covers every screen until timer start or sign-out. [S1, pp. 5-6, 9-10]

06 / TARGET SYSTEM

Updated architecture



PROPOSED. Preserve the stack and one backend initially. Separate business modules within that service; introduce no Redis, Kafka, Kubernetes, microservices or additional paid server without measured need. [S2]

Responsibilities

Desktop: local idle detection, local UI state, indexed durable cache, bounded sync queue and update manager. Dashboard: on-demand reports, pagination, date filters and conditional visible-tab refresh. API: authentication / RBAC, timers / attendance, catalogs, billing / reports, sync / cache and health / metrics. MongoDB: pooled connections, query-specific indexes, projections, safe bulk operations and restore-tested backups.

Boundaries

Keep authorization and accepted time calculations on the server. Treat queued client signals as untrusted inputs. Retain timer-lock.js and the full-screen lock behavior during restructuring even though the supplied target tree is illustrative. The complete proposed folder tree appears unchanged in Appendix A; it is a migration target, not a repository inventory.

07 / RENDER ANALYSIS

Separate the two traffic problems

REPORTED CATEGORY	WHAT IT MEANS	INVESTIGATION
4.8 GB service-initiated	Render service outbound traffic to public destinations, across protocols. [R1, R2]	Measure bytes sent to Atlas and external HTTP / SMTP destinations, retries, connections and jobs.
1.92 GB HTTP responses	Outbound replies to browser / desktop HTTP requests. [R1, R2]	Rank endpoints by compressed bytes, count, status, report range, static assets and cache outcomes.

The arithmetic total is 6.72 GB. Service-initiated is approximately 71.4% and HTTP responses 28.6% of that sum. The ratio is 2.5:1. These percentages are valid only if the figures share a period and scope and are non-overlapping categories in the same view. No cost estimate is made without the plan, included allowance and billing scope.

Database direction matters

For the Render service, query commands, writes and protocol traffic sent to an external Atlas destination are outbound candidates. Database result bytes received by the backend are inbound to that service; do not automatically count every fetched database byte as Render outbound. Smaller reads can still reduce response sizes and workload. Correlate direction-specific transport metrics before attributing savings. [R1, R2]

Candidate causes - hypotheses, not findings

• Repeated heartbeat writes, session operations or large update payloads sent to Atlas; repeated connection setup or aggressive reconnect loops.

• External APIs, mail, storage, jobs or self-calls. Check destinations and direction; repeated logs printed to stdout do not prove billed transfer.

• Repeated history / catalog / dashboard responses contributing to HTTP bytes. Existing compression and conditional caching may already limit them.

Updater attribution
The source hosts desktop updates on GitHub Releases. Installer downloads from GitHub are not evidence of bytes served by Render. Attribute them to Render only if the repository shows a proxy or Render-hosted artifacts.

Render documents hourly bandwidth points and about a one-hour reporting delay. Compare completed, aligned windows and allow for that lag. [R2]

08 / POLLING MATH

Request volume and response sensitivity

CALCULATED. Assumptions: 15 employees, 8 active hours per day, 22 working days; one heartbeat and one refresh request per cycle. This is a scenario, not the actual October working calendar or a measured request count.

STREAM	PER EMPLOYEE / DAY	ALL / DAY	22 DAYS
20 s heartbeat	28,800 / 20 = 1,440	21,600	475,200
30 s refresh	28,800 / 30 = 960	14,400	316,800
Combined	2,400	36,000	792,000
60 s policy (extra)	480	7,200	158,400
One 8-hour dashboard (extra)	960	960	21,120

If the 30-second refresh makes k network calls, its monthly total is 316,800 × k. General total for the two core loops = 15 × 8 × 3,600 × 22 × (1/20 + k/30). Do not count the five-second local idle check as an API call unless code confirms a transmission.

AVG RESPONSE BODY	792,000 RESPONSES	ASSUMPTION
0.25 KB	0.198 GB	Decimal KB = 1,000 bytes
1 KB	0.792 GB	Source estimate: approximately 0.79 GB
2 KB	1.584 GB	Body bytes only
5 KB	3.960 GB	No savings prediction
10 KB	7.920 GB	Illustrates sensitivity to payload size

1.92 GB / 792,000 ≈ 2,424 bytes per response, if all reported HTTP bytes belonged to this modeled workload. That conditional average does not prove actual response size: the measurement period, extra calls, static assets, compression and header accounting are unknown.

Savings scenario
If 100,000 measured responses shrink from 10 KB to 1 KB, response-body savings are 0.90 GB (decimal). This is an illustration, not a forecast. A 304 removes the representation body but still has request and response overhead. Keep the 20-second heartbeat initially.

09 / PRIORITIES

Optimization backlog and measurements

PRIORITY	CHANGE	EVIDENCE / SUCCESS CHECK
P0	Instrument endpoint and outbound bytes	Match scope and period; rank destinations and endpoints; redact secrets; no invented attribution.
P0	Preserve timer correctness and recovery	Race / retry / crash tests pass; no timer, attendance or billing drift.
P1	Delta attendance / timesheet sync	Edits and deletions arrive; cursor advances only after commit; reconciliation matches baseline.
P1	Catalog cache and validation	Audit existing 304 behavior; correct invalidation after changes and permission updates.
P1	Paginate and date-filter reports	Bound pages / ranges; compare full totals and billed amounts with baseline.
P1	Optimize MongoDB queries	Use explain on actual filters and sort; reduce examined documents / returned fields without lost rows.
P1	Deduplicate requests / polling	One in-flight request per scope; no overlapping loops; appropriate cancellation and retry.
P2	Backups and restore tests	Demonstrate recovery in an isolated database; document retention and recovery objectives.
P2	Operational alerts / health	Detect API errors, stale timers, reconnect storms, growing sync backlog and category byte increases.

Measurement design

Record route template, method, status, count, request / response bytes, duration, encoding and cache result. Use bounded-cardinality destination labels for outbound traffic and command counts / latency for MongoDB. Do not log raw employee data, authorization headers, tokens or passwords. App body counters do not equal TLS / TCP wire measurements or Render billing; use them to rank and explain, then reconcile with provider totals.

Capture a representative baseline, then change one area at a time. Normalize before / after by employee-hours and workload. Track response bytes per request, outbound bytes per active hour, database commands per heartbeat and p95 latency. Set numerical savings targets only after attribution; correctness gates apply regardless of savings.

10 / TIMER RELIABILITY

Keep timing rules coupled

SOURCE-DOCUMENTED. Heartbeat every 20 seconds; stale threshold 90 seconds; recovery scan every 30 seconds; recovery closes at the last heartbeat. The backend is the clock. Idle stop is five minutes and logout fifty minutes, with policy overrides as documented. [S1, S2]

Timing envelope

CALCULATED. With a regular scan and no scheduler delay, stale detection occurs approximately 90 to 120 seconds after the last accepted heartbeat, depending on scan phase. Real event-loop or database delay can extend it. This is detection latency, not a rule to bill until detection: the source says close at the last heartbeat. Verify strict versus inclusive comparisons and all endpoint rounding rules in code.

Safe operation design - proposed

• Start, pause and stop use an operation ID / idempotency key scoped to employee and session. Replay returns the original outcome; a reused key with different content is rejected. Persist the result so process restart does not lose deduplication.

• Enforce one running timer and one active desktop session using verified database constraints plus conditional writes. Handle duplicate-key conflicts deterministically; a pre-check alone is insufficient under races. [M1]

• Resolve stop / heartbeat / recovery races with state or version predicates. A late heartbeat cannot reopen a stopped timer. Recovery updates both time log and attendance consistently, using a verified transaction or replayable repair mechanism.

• Keep accepted timestamps authoritative. Client idleSeconds and queued timestamps require bounds, session ownership and sequence validation. Distinguish local monotonic elapsed time from wall-clock display; reject impossible future or out-of-order events.

• After sleep, crash or network loss, reconcile server state before retrying operations. Retain an audit trail for disputed time and manual corrections. Offline accumulation must follow existing business rules; do not silently invent payable time after a recovered stop.

Heartbeat interval restriction
Do not increase the heartbeat interval as a first optimization. Review the lease threshold, recovery scan, jitter, missed-heartbeat tolerance, revocation latency and attendance behavior together, then test a documented proposal before any change.

11 / SYNC & DATABASE

Bounded traffic without missing records

Incremental sync contract - proposed

Return authorized changes in stable pages using a server-issued cursor and deterministic tie-breaker (for example updatedAt plus ID, if that schema supports it). Bind cursors to employee, permissions and filter scope. Include deletions via tombstones or a compatible change mechanism. Use an upper watermark so a paginated sync has a stable boundary; preserve concurrent later updates for the next cycle.

Apply a page and save its cursor atomically in the local cache. Retry safely after interruption. Expired cursors, changed permissions, new devices and suspected divergence trigger bounded full reconciliation. Clear sensitive cache data on logout or account switch. Reconcile active timers separately from historical rows, without trusting stale cached timer state.

Conditional catalogs and reports

Audit the existing 304 list responses before adding ETags. Validators must reflect authorization scope and all fields affecting the representation; invalidate after edits and role changes. Review Cache-Control and conditional-request behavior together because “never cached by the browser” and “304 when unchanged” require a precise contract. Keep authenticated responses private; use no-store for highly sensitive material where appropriate.

Filter reports by authorized employee / team and date range before pagination. Use stable sorting and bounded page size. Compute aggregates on the server for the entire filter set rather than summing one page. Distinguish record changes from totals changes. Refresh on focus or explicit action with a defined freshness policy; preserve current visible-tab behavior.

MongoDB query and write review

Reuse a bounded process-level pool; do not reconnect on every request. Measure filters, projections, populate chains, examined documents, sort stages and write sizes. Candidate index families include employee + date, employee + updatedAt + ID and active timer status + heartbeat; these are hypotheses to verify with actual queries, not create-index instructions.

Preserve existing unique constraints. Partial unique indexes constrain only matching documents; confirm actual predicates, state values and existing duplicates before migration. Indexes add write and storage cost. Bulk only operations with compatible ordering and failure semantics; heartbeat persistence must still support the agreed recovery accuracy. [M1]

12 / SECURITY & OPERATIONS

Preserve access and safe recovery

Session and authorization controls

Validate JWT signature, allowed algorithm, expiry and intended issuer / audience as applicable. Enforce active account and current session state. Preserve one active desktop session and second-device approval: new login returns 409, both sides long-poll, current device decides, approval signs in the new device and revokes the old; expiry after two minutes. Browser sessions are not limited in the same way. Verify real route prefixes and abort orphaned long-polls. [S1, pp. 6-7]

The source sign-in cache is 15 seconds and role permissions apply within 30 seconds. Document and test maximum revocation delay; invalidate caches on explicit revocation and critical changes where possible. Check record ownership and team scope on every endpoint, including sync, billing, exports and manual-time edits. Do not infer access rights from hidden buttons or CORS.

Electron hardening - proposed review

Use context isolation, renderer sandboxing, a narrow preload bridge and validated IPC senders / arguments. Keep Node integration disabled for remote content; restrict navigation, new windows, external URLs and permissions. Apply a restrictive content security policy, maintain supported dependencies and protect secrets in OS-backed storage. Sign update artifacts and verify publisher / source; prevent updates while a timer runs and verify rollback / failed installation behavior. [E1]

Operational reliability

On shutdown, stop accepting new work, drain bounded in-flight requests and close connections cleanly. Timer recovery must be safe to rerun after restart. Monitor recovery-job lateness, duplicate sessions, pending approvals, retry storms, sync backlog, API errors, database latency and traffic anomalies. Health checks should reveal no secrets. Long-poll and request timeouts must be bounded.

Define backup schedule, retention, encryption, access controls and recovery objectives with the owner. Test restore into an isolated environment, including indexes and migrations. Document deployment and rollback steps plus compatibility with older desktop versions. Overnight shifts and timezone boundaries must retain existing attendance-day and billing rules; changing them requires an explicit business decision.

13 / MIGRATION

Phased rollout with exit gates

PHASE	IMPLEMENTATION	EXIT / ROLLBACK
0. Audit and baseline	Inventory clients, endpoints, polls, routes, models, constraints, external calls and existing optimizations. Add redacted metrics.	Publish confirmed issues and hypotheses; capture baseline. Remove or disable instrumentation if overhead is material.
1. Low-risk traffic fixes	Deduplicate calls, cancel obsolete work, tighten projections and validate existing caches / report filters.	Business results unchanged; errors and normalized transfer do not regress. Revert flags per change.
2. Versioned sync	Add cursor endpoints beside current endpoints; implement local cache migrations and reconciliation.	Shadow-compare IDs, values, deletes and totals; pilot clients; fall back to old reads without losing queued operations.
3. Timer / session safety	Add verified idempotency, conditional transitions and recovery invariants; retain timings.	Race, retry, sleep, restart and revocation suite passes. Replay-safe rollback without duplicate billing.
4. Modular migration	Move one business module at a time into target structure, with adapters preserving routes and contracts.	Relevant tests pass after each move. Revert code moves separately from schema changes.
5. Deployment and operations	Pilot a small group, expand to 15 employees, observe complete aligned windows; document backups and alerts.	Correctness gates pass; measured savings attributed by category. Roll back on time / permission drift or persistent errors.

Database and client compatibility

Use expand / migrate / contract changes and versioned migrations. Validate indexes against existing data, back up before structural changes and rehearse recovery. Keep old clients functional during rollout; never require every employee to upgrade at once for basic timer operations. Preserve installer release metadata, screen locks, updater behavior and all existing business rules.

The full source Codex prompt in Appendix A requests an audit report and implementation plan first, before modifying production-critical functionality. This report is that planning input; it does not claim implementation or production verification.

14 / VERIFICATION

Regression matrix and acceptance criteria

TEST GROUP	REQUIRED SCENARIOS / PASS CONDITION
Timer transitions	Start / pause / stop / duplicate retries; concurrent devices; heartbeat racing recovery; late heartbeat after stop. Exactly one accepted running timer and stable totals.
Fault recovery	Offline, sleep, abrupt app kill, server restart, database reconnect and shutdown. No reopened stopped timer, lost acknowledged operation or duplicate attendance update.
Time boundaries	Overnight shift, timezone / daylight-saving boundary where applicable, midnight, clock skew and idle policy overrides. Totals match existing approved rules.
Attendance / billing	First start, last stop, breaks, idle deduction and authorized manual edits. Compare employee, team and company reports and billing totals with baseline.
Sync / cache	Concurrent edits, delete tombstones, cursor expiry, page retry, account switch, role change and reconciliation. No missing / duplicated records or unauthorized cache reuse.
Permissions / sessions	Each role, personal grant, cross-employee ID access, revoked token, inactive account, approval / rejection / 2-minute expiry and long-poll cancellation. Server enforces scope.
Reports / database	Bound filters and stable pagination; whole-filter totals unchanged; explain plans reviewed; constraints survive migration / restore.
Desktop / updates	All-monitor lock, idle stop / logout, close while running, failed update, signed update, safe restart and retained login. Existing behavior preserved.
Transfer / performance	Before / after endpoint and destination totals on comparable workload; compressed-body and wire layers identified; Render reporting lag accounted for.

Release acceptance

Run existing automated tests, then targeted regression and integration checks against a test database. Require no unexplained change to time, attendance, billing or permission results; no data loss; successful restore drill; and a rehearsed rollback. A bandwidth reduction cannot compensate for failed correctness checks. Record test evidence, changed files, migrations and known limitations for every phase.

15 / REFERENCES

Provenance and preservation record

S1 | Supplied source

handdy archeture.pdf
Original 10-page image-based architecture source, dated 26 October 2026; documented desktop version 1.2.32.

S2 | Supplied source

Pasted markdown(4).md
Full optimization proposal, target folder structure and full Codex prompt; reproduced in Appendix A.

R1 | Official documentation

https://render.com/docs/outbound-bandwidth
Render outbound bandwidth definitions and directionality.

R2 | Official documentation

https://render.com/docs/service-metrics
Render HTTP response / service-initiated category definitions and hourly reporting behavior.

M1 | Official documentation

https://www.mongodb.com/docs/manual/core/index-partial/
MongoDB partial indexes and partial unique constraints.

E1 | Official documentation

https://github.com/electron/electron/blob/main/docs/tutorial/security.md
Electron security guidance, maintained by the Electron project.

Public documentation checked 26 October 2026. No provider pricing, exact installed dependency versions or hosting plan is inferred from these references. The architecture source records version labels; these have not been validated against a repository.

What is delivered and verified

Part I provides readable diagrams, analysis, calculations, reliability design, migration gates and regression requirements. Appendix A contains the complete Markdown source, including its full tree and four-phase Codex prompt. Appendix B appends all ten original pages without altering their contents. Both original files are also embedded as PDF attachments, so exact source bytes remain available.

Source clipping remains visible in the preserved original pages. That limitation belongs to the supplied PDF; no missing right-edge wording has been fabricated. The new pages are checked separately for layout, content coverage, arithmetic and readable typography. Live system correctness and actual savings require repository tests and measured rollout evidence.

APPENDIX A / COMPLETE SOURCE

Updated CRM Architecture - source text

Complete Pasted markdown(4).md. The original wording and order follow, including all recommendations, the full folder tree and the complete Codex prompt. Source assertions remain subject to the evidence labels and clarifications in Part I.

Updated CRM Architecture — Optimized for Low Render Data Usage

For your 15 employees, 7 GB of Render data transfer since the beginning of October is worth investigating, especially because your CRM mainly handles attendance, timers, tasks, and reports.

Based on your architecture PDF, I recommend keeping your existing technology stack but changing how the desktop app communicates with the server.

1. What is causing the high data usage?

Your reported metrics are:

Render usage reported

6.72 GB

Combined reported categories, approximately 7 GB

Service-initiated

4.8 GB

HTTP response

1.92 GB

These figures are your reported Render metrics. The categories should be verified against Render's definitions before treating them as distinct billable transfer totals.

Your PDF shows several possible sources of unnecessary traffic:

- Desktop heartbeat every 20 seconds.

- Desktop session and timesheet refresh every 30 seconds.

- Client, project and task list refreshes.

- Dashboard refresh every 30 seconds.

- Server database reads and writes for active timers.

- Repeated session validation and cache refreshes.

Important: The PDF does not establish which of these accounts for the 4.8 GB service-initiated traffic. That could also involve outbound database connections or other server-to-server requests. We need traffic measurements to identify the actual cause.

2. Recommended architecture

Electron Desktop — 15 Employees

Local idle detection • Local UI state • Indexed local cache • Batched sync • Update manager

React Admin Dashboard

On-demand reports • Paginated records • Conditional refresh

HTTPS • Compact JSON • Delta sync • ETag

Node.js + Express Backend

Render — single service initially

Authentication & RBAC

Timer & Attendance

Tasks & Clients

Billing & Reports

Sync & Cache

Health & Metrics

Pooled connections • Indexed queries • Bulk writes

MongoDB Atlas

13 existing collections • Proper indexes • Backup policy

Operational monitoring

Structured Logs + Metrics + Alerts

Requests • Response bytes • Database latency • Timer errors

This design does not require Redis, Kafka, Kubernetes, microservices, or an additional paid server for 15 employees.

3. Changes I recommend implementing first

Priority	Change	Why
P0	Measure outgoing traffic by endpoint and destination	Find the real source of 7 GB
P0	Preserve timer accuracy and recovery	Prevent incorrect employee timesheets
P1	Delta sync for timesheets and attendance	Avoid repeated full-history downloads
P1	Cache clients, projects and tasks	Avoid downloading unchanged data
P1	Add pagination and date filtering	Reduce large report responses
P1	Optimize MongoDB reads and indexes	Reduce latency and database traffic
P1	Stop duplicate requests and overlapping polling	Reduce wasted network activity
P2	Add scheduled backups and restore testing	Protect historical billing records
P2	Add alerts and deployment health checks	Improve long-term reliability

Do not simply increase the heartbeat interval or disable background checks to save bandwidth. Your existing 90-second timer recovery mechanism depends on heartbeats, so changing one interval without the others can incorrectly stop active timers.

4. Updated folder structure

I recommend restructuring the backend by business functionality rather than keeping every feature mixed together.

```
CRM/
├── backend/
│   ├── src/
│   │   ├── config/
│   │   │   ├── database.js
│   │   │   └── environment.js
│   │   ├── modules/
│   │   │   ├── auth/
│   │   │   ├── employees/
│   │   │   ├── clients/
│   │   │   ├── projects/
│   │   │   ├── tasks/
│   │   │   ├── timers/
│   │   │   ├── attendance/
│   │   │   ├── billing/
│   │   │   ├── reports/
│   │   │   └── permissions/
│   │   ├── services/
│   │   │   ├── sync.service.js
│   │   │   ├── timer-recovery.service.js
│   │   │   └── cache.service.js
│   │   ├── middleware/
│   │   ├── jobs/
│   │   ├── monitoring/
│   │   └── server.js
│   └── tests/
├── dashboard/
│   └── src/
│       ├── api/
│       ├── features/
│       ├── hooks/
│       └── components/
├── desktop-app/
│   ├── main.js
│   ├── preload.js
│   ├── renderer.js
│   ├── services/
│   │   ├── local-cache.js
│   │   ├── sync-manager.js
│   │   └── timer-manager.js
│   └── app-updater.js
└── shared/
    └── api-contracts/
```

This is a target structure, not a claim about the current repository. Migrate incrementally rather than moving all files at once.

5. How to reduce the Render transfer

A useful starting point is to estimate how often your desktop applications make requests.

Your PDF describes a heartbeat every 20 seconds and a refresh every 30 seconds.

For 15 employees working 8 hours daily:

Request	Approximate requests/day
Heartbeat, every 20 seconds	21,600
Refresh, every 30 seconds	14,400
Combined	36,000

That is approximately 792,000 requests over 22 working days, before counting dashboard traffic and other calls.

A large number of requests does not automatically mean high bandwidth. For example, 792,000 responses averaging 1 KB would transfer approximately 0.79 GB of response bodies. The actual total also depends on headers, request bodies, compression, other endpoints, and outbound connections.

I recommend these specific changes:

1. Keep the 20-second heartbeat initially, but make its payload minimal and avoid writing unchanged values unnecessarily.

2. Change timesheet synchronization to fetch only records updated since the last successful synchronization, with a safe full-reconciliation fallback.

3. Load clients, projects and tasks when needed, and use ETags or version numbers to avoid repeatedly transferring unchanged lists.

4. Fetch reports only for the selected date range and page. Do not download all historical records to calculate one screen.

5. Deduplicate requests on the desktop and dashboard, cancel obsolete requests, and avoid overlapping polling.

6. Record request counts, bytes sent, bytes received, status codes, database query times, and outbound destinations before and after each optimization.

For MongoDB, review compound indexes around employee, date, task, active timer status, and synchronization timestamps. Verify them against actual query patterns rather than creating indexes blindly.

6. Long-term reliability improvements

The most important design rule is that the backend remains authoritative for time calculations, while the desktop supplies activity signals.

For production, I would also require:

- Atomic timer start, stop and pause operations with idempotency keys.

- Reliable recovery when an employee loses internet connectivity or the desktop crashes.

- Clear handling of overnight shifts and timezone boundaries.

- Versioned database migrations and rollback procedures.

- Automated backups with tested restoration.

- Graceful shutdown and restart handling on Render.

- Electron security hardening, signed updates, and secure token storage.

- Automated tests covering simultaneous sessions, timers, attendance, billing and report totals.

Your existing 13 MongoDB collections can remain. There is no reason to replace MongoDB solely because Render transfer is high.

7. Prompt to give Codex for restructuring

You can paste this into Codex after opening your CRM repository.

Review my existing Handdy/CRM repository and improve its architecture for long-term production use.

Context:

- Approximately 15 employees use the Electron desktop application.
- React dashboard for employees, team leads, HR, managers and administrators.
- Node.js/Express backend hosted on Render.
- MongoDB Atlas with Mongoose.
- Render has reported approximately 4.8 GB service-initiated transfer and 1.92 GB HTTP response transfer since the beginning of October.
- The current architecture uses desktop heartbeats every 20 seconds, periodic timesheet synchronization and dashboard refreshes.

Primary objective:
Reduce unnecessary network transfer, database queries and server resource consumption without breaking any existing CRM functionality.

Phase 1 — Audit before changing code

1. Inspect the complete repository and identify all frontend, desktop and backend API calls.
2. Identify polling intervals, duplicate requests, unnecessary loops and full-history downloads.
3. Review MongoDB queries, indexes, connection pooling and caching.
4. Inspect timer, attendance, session and billing calculations.
5. Identify external service requests that could contribute to Render service-initiated traffic.
6. Add safe request and response byte measurements without logging passwords, tokens or sensitive data.
7. Produce an audit report listing confirmed issues, suspected issues and recommended fixes.

Phase 2 — Architecture improvements

1. Preserve the existing Node.js, Express, React, Electron and MongoDB stack.
2. Improve module separation between routes, controllers, services and database models.
3. Introduce incremental timesheet synchronization with a safe reconciliation mechanism.
4. Use ETags or version-based caching for clients, projects, tasks and settings.
5. Add pagination and server-side date filters for large reports.
6. Deduplicate concurrent API calls and eliminate unnecessary polling.
7. Optimize MongoDB queries using appropriate indexes and projections.
8. Ensure timer operations are atomic and idempotent.
9. Preserve server-authoritative timekeeping and correct attendance calculations.
10. Add structured monitoring for API traffic, errors and database performance.

Phase 3 — Reliability and security

1. Review JWT validation, session revocation and role permissions.
2. Protect against unauthorized access to employee records.
3. Preserve the single-active-desktop-session policy and second-device approval workflow.
4. Preserve idle tracking, automatic timer stopping, overnight shifts and desktop automatic updates.
5. Improve backend restart recovery and database error handling.
6. Document backup, restore, deployment and rollback procedures.

Phase 4 — Verification

1. Run existing automated tests.
2. Add regression tests for timer, attendance, billing, permissions and synchronization.

3. Verify that optimized APIs return the same business results as before.

4. Compare request counts, response sizes, database operations and estimated transfer before and after changes.

5. Document every file changed and any migration required.

Important restrictions:

- Do not rewrite the entire application unnecessarily.

- Do not change existing business rules.

- Do not delete existing data.

- Do not remove any existing functionality.

- Do not introduce Redis, microservices, Kubernetes or additional paid infrastructure without proving a need.

- Do not increase heartbeat intervals until the timer recovery logic has been reviewed.

- Do not assume that HTTP polling is the confirmed cause of the 4.8 GB service-initiated transfer.

- Make changes incrementally and verify each phase.

Start with the audit report and implementation plan. Do not modify production-critical functionality until the impact has been identified.

Final recommendation

For 15 employees, keep your single Render backend and MongoDB architecture. Optimize and measure before upgrading hosting or adding services.

The first question I would investigate is why service-initiated transfer is 4.8 GB, substantially more than your reported HTTP response transfer. That distinction is essential: reducing desktop polling may improve HTTP traffic without fixing the largest category.

Your architecture PDF explains the system design, but it cannot prove which code paths generate those bytes. The next step is to compare Render metrics with actual backend request logs and outbound traffic measurements. That will let us restructure the code based on evidence rather than guesses.

APPENDIX B / ORIGINAL SOURCE

Original architecture PDF

The following ten pages reproduce handdy archeture.pdf in full, in the original order. Original page labels remain 1/10 through 10/10. Report page numbering continues from this divider in the outer margin.

Preserved source limitation
The original PDF is image-based. Its diagram and table containers were already horizontally clipped when printed. These pages deliberately retain that state. Use the readable reconstructions in Part I for architecture discussion and the attached original for source-level review.

Original source fingerprints

handdy archeture.pdf

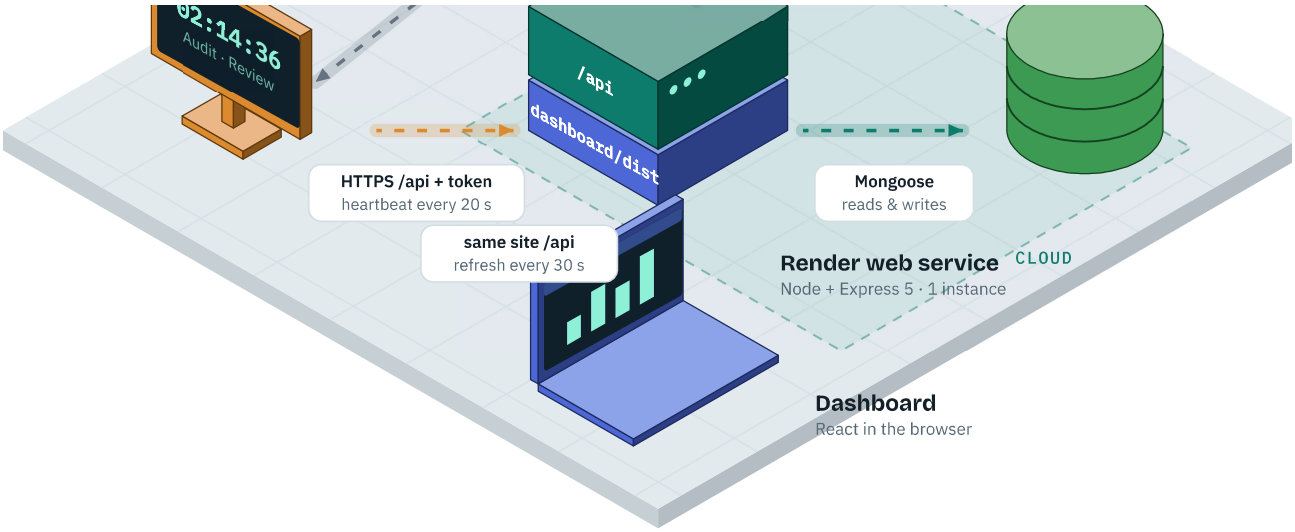
SHA-256: 677f83f6e798eaea640c44256feffca80fda6ccd1ac30048e5f0eccdec76d909

Pasted markdown(4).md

SHA-256: 61631d02f6ea701dae801ed3061d2b754335e682936bd42423b55f332256f5f7

Source boundaries

The original source describes the application as drawn from the repository on 26 October 2026. That is a claim made by the source author. This report has not independently audited the repository, schema, deployment, session implementation or billing account. Proposed safeguards and candidate optimizations must be validated during the audit phase.


02:14:36
Audit · Review
/api
dashboard/dist
HTTPS /api + token
heartbeat every 20 s
same site /api
refresh every 30 s
Mongoose
reads & writes
Render web service
CLOUD
Node + Express 5 · 1 instance
Dashboard
React in the browser

Arrows point from the side that starts the connection. Both apps call the same API with a sign-in token; only the server reads and writes the database. GitHub feeds the server (on every push to `main`) and the desktop app (through Releases).

Desktop app

`desktop-app/ · Electron 44`

The only place time is recorded. Sign in, pick Project → Client → Task, start the timer. Stops the timer after 5 minutes without input, signs out after 50, and locks the screen while no timer runs.

Calls `https://handdy-workspace.onrender.com/api`

Dashboard

`dashboard/ · React 19 + Vite 8`

Web pages for admins, managers, HR, team leads and employees: reports, attendance, billing, tasks, teams, clients and permissions. Menus and buttons follow each role's permissions.

Calls `/api` on the same website

Server

`backend/ · Node + Express 5`

Holds every business rule and security check, works out time totals, records attendance, closes timers whose PC went silent, and serves the dashboard pages.

Runs on Render, one instance, health check `/health`

Database

`MongoDB Atlas · Mongoose 9`

13 collections: employees, teams, clients, projects, tasks, time logs, attendance, sessions and settings. Nothing else connects to it.

Only the server connects

REQUEST PATH

What the server does with every call

The same path runs for the desktop app and the dashboard. A request is refused at the first step it fails; the red notes show what the caller then receives.

01	**Arrives over HTTPS at `/api/…`** The app sends `Authorization: Bearer <token>`, issued at sign-in.	
02	**Compression and JSON body**	`413 too large`

	Bodies over 1 MB are refused.	
03	**NoSQL guard** Removes $ and dotted keys so nobody can inject database operators.	
04	**Security headers and CORS** Only the dashboard's own site and the desktop app's pages may call from a browser.	
05	**Router picks the file in `routes/`** Each route lists its access rule next to it.	`404 unknown path`
06	**Sign-in check (`protect`)** Token valid, account active, session still active. Cached for 15 s to save database reads.	`401 SESSION_REVOKED` · `403 ACCOUNT_INACTIVE`
07	**Role or permission gate** `authorizeRoles` for role-only areas, `requirePermission` for switches on the Permissions page.	`403 no permission`
08	**Controller does the work** With helpers in `services/`: timer lease, attendance, sessions, caches, report sync.	`400 / 409 rule broken`
09	**Model reads or writes MongoDB** Mongoose schemas in `models/`; unique indexes stop a second running timer or desktop session.	
10	**JSON reply** Never cached by the browser. Client, project and task lists answer `304` when nothing changed.	

API MAP

Every area of the API and who calls it

All paths start with `/api`. "Who can call" is checked by the server, whatever the

screens show.

AREA	ENDPOINTS	CALLED BY	WHO CAN CALL
Sign-in & sessions	`/auth/login` `/auth/quick-login` `/auth/me` `/auth/logout` `/auth/login-requests/…`	Desktop Dashboard	Anyone to sign in tries per 15 min) signed-in users f rest
Passwords	`/auth/forgot-password` `/auth/reset-password` `/auth/change-password`	Dashboard	Anyone (5 tries p min); reset email SMTP on Render
Employees & teams	`/employees` `/employees/options` `/employees/my-team` `/teams` `/auth/register`	Dashboard	Admins; HR for li teams; team lead their team; peop "create/edit emp
Clients, projects, tasks	`/clients` `/projects` `/tasks` `/tasks/bulk`	Desktop Dashboard	Everyone reads (needs them to st timer); changes r Create/edit clien projects or task permissions
Timer	`/time-logs/start` `/time-logs/heartbeat` `/time-logs/stop` `/time-logs/active` `/time-logs/note` `/time-logs/history`	Desktop	The signed-in em for their own tim
Reports	`/time-logs/admin` `/time-logs/team` `/time-logs/live` `/time-logs/history`	Dashboard	View all employe reports (compan View team times (team leads) · ow history for everyo
Manual entries	`/time-logs/manual` `/time-logs/manual/options` `/time-logs/manual/:id`	Dashboard	Admins, manage HR for every emp team leads for th

			team when grant

Attendance	`/attendance/admin/history`
	`/attendance/team/history`
	`/attendance/history`
		Dashboard	Admins, manage team leads for th team · everyone themselves. Row written by the tin by a button.

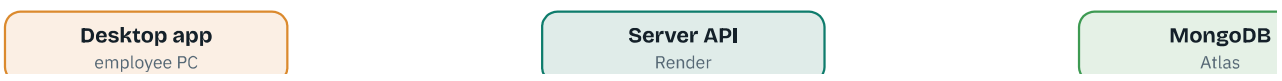
Billing	`/billing/employees` `/billing/report`	Dashboard	View billing amou (admins, manage
Idle & logout settings	`/settings/idle-policy` `/settings/employee-idle-policies`	Desktop Dashboard	Everyone reads t policy; admins ch alpha sets per-pe rules
Permissions	`/permissions/me` `/permissions` `/permissions/roles/:role`	Dashboard	Everyone reads t own; admin and change roles
Shifts & schedules	`/shifts` `/schedules`	Dashboard	Admins, manage (pages exist, not menu)
Health	`/health` (no `/api`)	Render	Public; Render us check the server

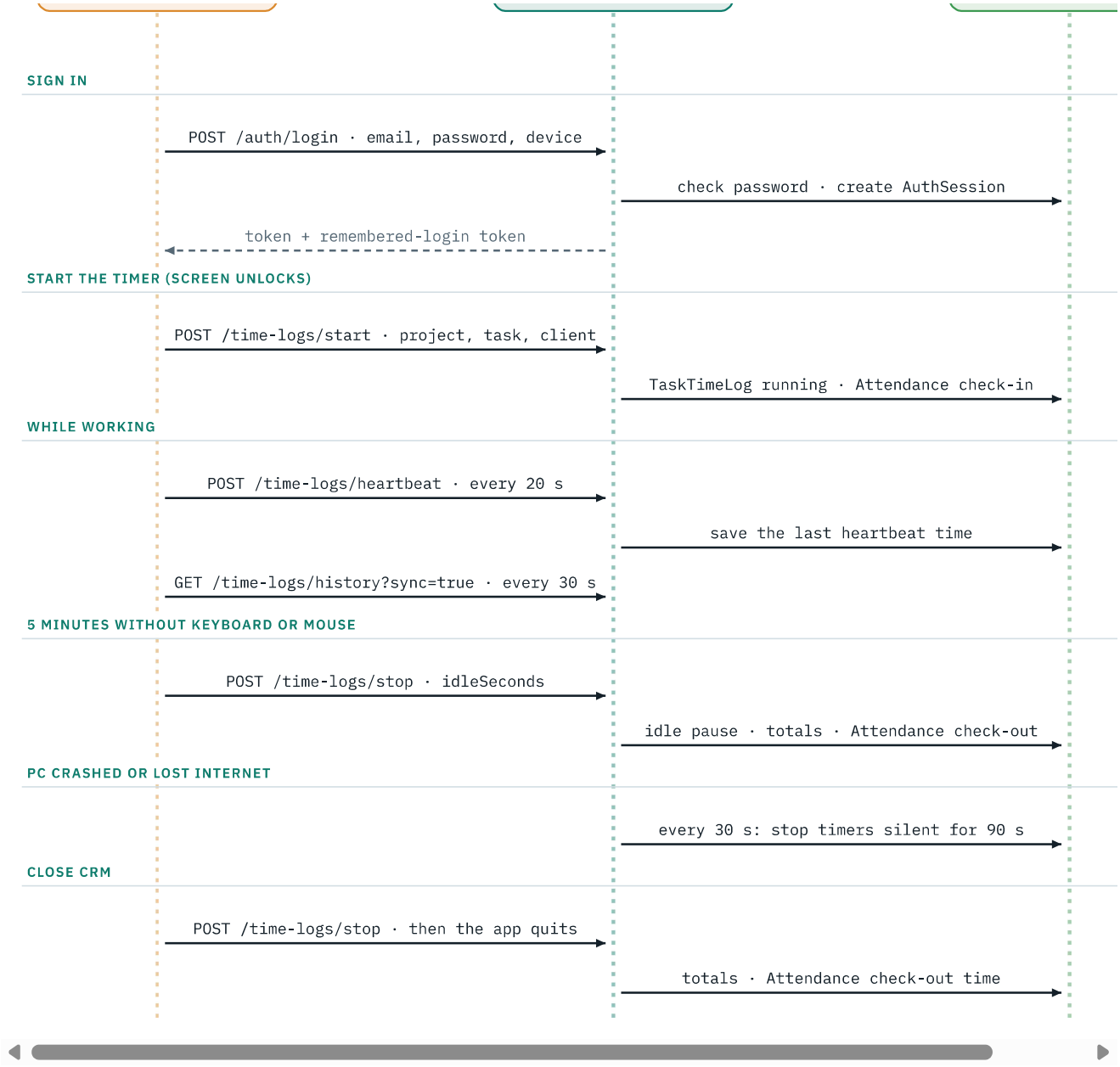
MAIN FLOW

An employee's day, call by call

The server is the clock: the desktop app only asks it to start and stop. Attendance follows automatically: the first start of the day is the check-in, the last stop is the check-out.

Swipe sideways to read every call →





Solid arrows are requests, dashed arrows are replies. While no timer runs, the desktop app covers every screen until the employee starts a timer or signs out.

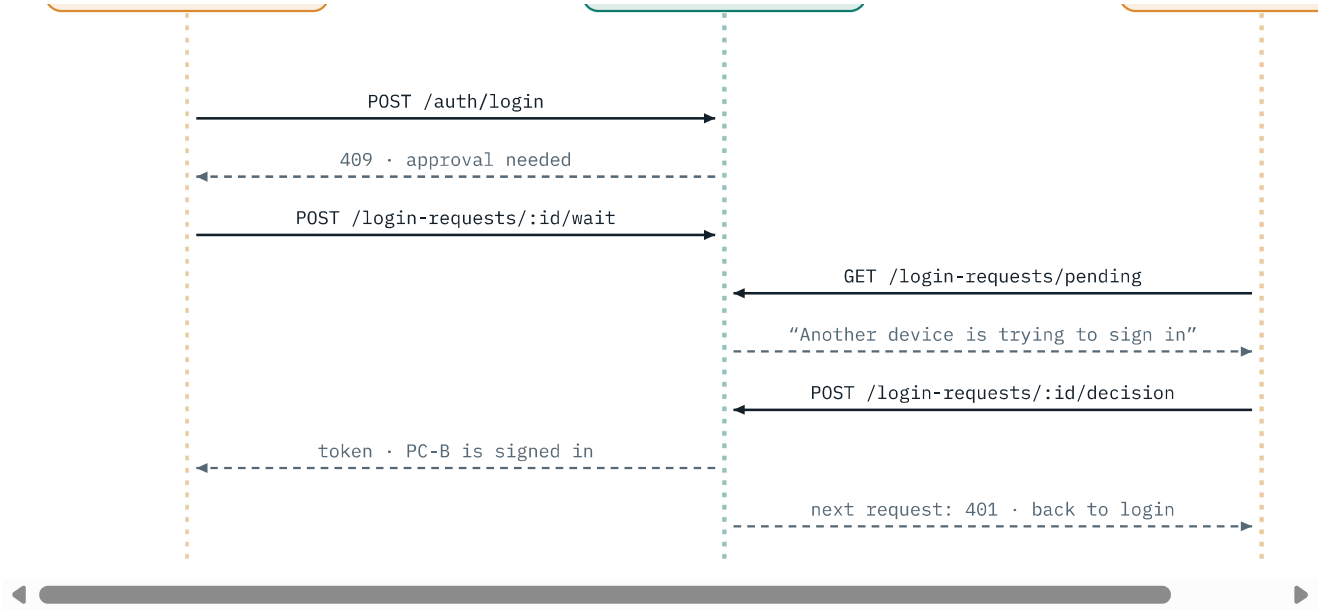
SECURITY FLOW

Signing in on a second computer

Each employee may have one active desktop session. A new computer has to be approved on the one already signed in. Browsers are not limited this way.

Swipe sideways to read every call →





Both computers wait with long-polls (requests the server answers as soon as something happens). If nobody decides within 2 minutes, the request expires.

SHIPPING CHANGES

How updates reach people

There are two separate routes. Changes to the server or dashboard need only a push. Only changes inside `desktop-app/` need a new installer.

Website and server

1. Commit and push to `main` on GitHub.
2. Render installs `backend` and `dashboard`, builds the dashboard and restarts the server.
3. Live within minutes. Browsers get the new pages on their next load; desktop apps use the new server straight away.

Desktop app

1. Raise the version in `desktop-app/package.json` and run `npm run build`.
2. Publish a new GitHub Release (tag `v1.2.33`…) with the `.exe`, `.blockmap` and `latest.yml`.
3. Each app checks 15 s after starting, then every 6 hours, and downloads quietly.
4. From 1.2.32 it restarts by itself to install once no timer runs and nobody has used the PC for a minute; otherwise it installs when CRM closes.

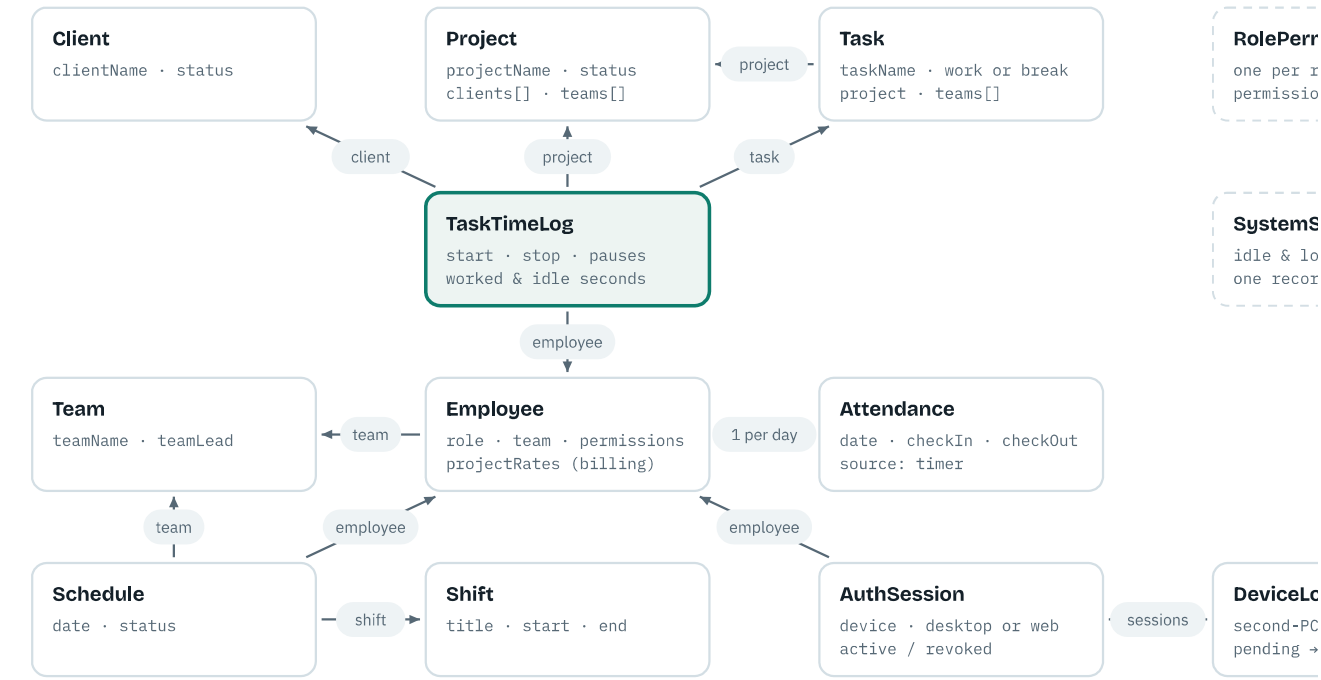
It reopens signed in.

STORED DATA

The 13 collections and how they link

Time logs sit at the centre: timesheets, reports, billing and attendance are all worked out from them.

Swipe sideways to see every collection →



Arrows point to the record that is referenced; for example, each time log stores its employee, project, task and client. Dashed boxes are stand-alone settings. A unique index allows only one running timer per employee and one attendance row per employee per day.

ACCESS

Who can do what

Full access

`alpha` `admin` `manager`

Set on the Permissions page

`hr` `team_lead` `employee`

Always hold every permission. Only admin and alpha edit the Permissions page; only alpha sets per-person idle rules.

Each switch says what it does and which roles it can apply to. The dashboard reads `/permissions/me` to hide menus and buttons; the server checks again on every call.

Three checks on the server

Role gates: areas tied to a role, such as team-only reports.

Role permissions: the switches, saved per role and applied within 30 s.

Personal grants: create or edit employees, given to one person.

BACKGROUND WORK

What runs on a timer

These repeat on their own. Most of the API traffic you see in logs comes from them.

WHERE	EVERY	WHAT IT DOES
Desktop	5 s	Reads keyboard/mouse idle time; idle stop at 5 min, sign-o 50 min; reports the screen-lock state
Desktop	20 s	Heartbeat that keeps the running timer alive on the server
Desktop	30 s	Checks the session and refreshes today's timesheet and th project, client and task lists
Desktop	60 s	Reloads the idle and logout policy
Desktop	15 s, 6 h	Update check after start, then every 6 hours
Desktop Dashboard	waiting	Long-poll for "another device is trying to sign in"

Dashboard	30 s	Refreshes the open page while its tab is visible (only chang rows for reports)
Server	30 s	Closes desktop timers with no heartbeat for 90 s, at the la heartbeat, and updates attendance
Server	15 s · 60 s	Sign-in cache · client/project/task list cache

REPOSITORY

Where the code lives

```
backend/
  server.js              app setup, middleware order, routes, startup
  src/routes/            URL → access rule → controller (start here for "who may cal
  src/controllers/       request handlers: timer, auth, reports, billing, manual ent
  src/middleware/        protect, role/permission gates, rate limits, catalog cache
  src/services/          timer lease, attendance, sessions, permission snapshot, rep
  src/models/            13 Mongoose schemas
  scripts/               admin tools: create alpha, repair teams, backfill attendanc
dashboard/
  src/routes/            every URL and its guard (role + permission)
  src/context/           sign-in state and permissions (useAuth)
  src/api/               one file per API area, all through axiosInstance
  src/pages/             screens by role: admin, teamLead, employee, shared
  src/components/        layout (menu, guards), reports, tasks, employees
desktop-app/
  main.js                window, safe close, screen lock wiring, update restart
  renderer.js            login, timer, idle, timesheet, lock state
  preload.js             the only bridge between the page and the system
  timer-lock.js          full-screen lock on every monitor
  app-updater.js         GitHub Releases updates
render.yaml            Render service: build, start, environment, 1 instance
CODEBASE_GUIDE.md      the full written guide
```

Drawn from the repository on 26 October 2026 (desktop 1.2.32). The written details are in